

MRM FINDING CLUSTERING SYSTEM

End-to-End Technical Documentation

Architecture · Mathematics · Code Walkthrough

Table of Contents

Table of Contents	2
1. System Overview	4
2. End-to-End Process Overview	4
3. Mathematical Concepts and Formulas	4
3.1 Term Frequency (TF) — Sublinear Scaling	4
3.2 BM25-Style Length Normalisation	5
3.3 Inverse Document Frequency (IDF).....	5
3.4 Final TF-IDF Weight	5
3.5 L2 (Euclidean) Normalisation.....	5
3.6 Cosine Similarity.....	6
3.7 Cosine Distance	6
3.8 K-Means++ Initialisation	6
3.9 K-Means Inertia	6
3.10 Bisecting K-Means.....	7
3.11 Silhouette Score	7
3.12 Gap Statistic	7
3.13 Composite k Score	7
4. Step-by-Step Process Flow.....	9
5. Detailed Line-by-Line Code Explanation	10
5.1 Imports and App Initialisation (Lines 1–7).....	10
5.2 In-Memory Data Stores (Lines 10–11).....	10
5.3 Stop Words (Lines 16–33).....	10
5.4 Field Weights (Lines 42–48).....	10
5.5 Theme Groups and normalize_theme (Lines 52–72).....	11
5.6 tokenize() — Text Normalisation (Lines 75–78).....	11
5.7 build_weighted_doc() — Field Repetition (Lines 81–94).....	11
5.8 build_tfidf() — Vectorisation Engine (Lines 97–138)	12
5.9 cosine() — Dot Product Similarity (Lines 141–142).....	12
5.10 kmeans() — K-Means with Restarts (Lines 145–207).....	13
5.11 bisecting_kmeans() — Hierarchical Splitting (Lines 210–270)	13
5.12 _silhouette_score() — Cluster Quality Metric (Lines 273–306)	14
5.13 _gap_statistic() — Random Baseline Comparison (Lines 309–352)	14
5.14 _auto_select_k() — Optimal k Decision (Lines 355–432).....	14
5.15 run_clustering() — Orchestration (Lines 488–543)	15
5.16 search_query() — Semantic Search (Lines 546–568)	15

- 5.17 Flask REST API Routes (Lines 572–751)..... 16
- 5.18 upload_findings() — File Import (Lines 626–714) 16
- 5.19 Seed Data and Application Startup (Lines 754–824) 17
- 6. Deployment Architecture..... 18
- 7. Summary 18

1. System Overview

This document provides a complete end-to-end technical explanation of an MRM (Model Risk Management) Finding Clustering System. The system ingests model audit findings, vectorises them using a custom TF-IDF engine, and groups them into semantically coherent clusters using Bisecting K-Means with automatic optimal-k selection. The result is a web application that allows users to upload, search, and explore findings by theme and cluster.

Technology Stack: Python (Flask) · NumPy-free custom ML · Pandas · OpenPyXL · Gunicorn · HTML/CSS/JS (single-page frontend)

Key Purpose: Automatically detect and group duplicate or thematically related MRM audit findings to reduce noise, surface themes, and aid reviewers.

2. End-to-End Process Overview

The system operates as a pipeline of discrete, well-defined stages. Each stage is described below with the inputs it consumes and the outputs it produces.

Stage	Input	Output
1. Data Ingestion	Raw finding records (JSON, Excel, CSV, or pre-seeded)	findings_db list of dicts
2. Weighted Doc Build	findings_db records	Composite weighted text strings per finding
3. Tokenisation	Composite text strings	Filtered token lists (stop-words removed)
4. TF-IDF Vectorisation	Token lists	L2-normalised TF-IDF vectors
5. k Selection (Auto)	Vectors + theme metadata	Optimal integer k
6. Bisecting K-Means	Vectors + k	Cluster label per finding
7. Cluster Assembly	Labels + findings_db	clusters_db list with metadata
8. API Exposure	clusters_db + findings_db	JSON REST API responses
9. Frontend Render	API JSON	Interactive web UI

3. Mathematical Concepts and Formulas

3.1 Term Frequency (TF) — Sublinear Scaling

Raw TF counts how many times a term appears in a document. However, a term appearing 100 times is not 100× more important than one appearing once. Sublinear scaling dampens this effect:

```
Sublinear TF(t, d) = 1 + log( count(t, d) )    if count > 0
                   = 0                        if count = 0
```

This compresses the range: a term appearing 10 times gets $TF \approx 3.3$ instead of 10. It prevents high-frequency domain jargon from overwhelming semantically distinct terms.

3.2 BM25-Style Length Normalisation

Longer documents naturally accumulate more term repetitions. To level the playing field between short and long findings, the TF value is further normalised using the BM25 length penalty:

```
TF_norm(t, d) = TF(t, d) / ( 1 - b + b × (|d| / avgdl) )
```

Where:

```
b      = 0.4  (length normalisation factor)
|d|    = number of tokens in document d
avgdl  = average document length across all findings
```

A $b=0$ disables length normalisation; $b=1$ is full normalisation. The chosen value of 0.4 is a moderate penalty — it prevents very long descriptions from drowning out short but precise titles.

3.3 Inverse Document Frequency (IDF)

IDF rewards terms that are rare across the corpus (discriminative) and penalises terms that appear in almost every document (generic). The smoothed formula used is:

```
IDF(t) = log( (N + 1) / (df(t) + 1) ) + 1
```

Where:

```
N      = total number of documents
df(t)  = number of documents containing term t
+1     = smoothing prevents division by zero
```

3.4 Final TF-IDF Weight

The final weight of a term in a document is the product of the length-normalised sublinear TF and the smoothed IDF:

```
w(t, d) = TF_norm(t, d) × IDF(t)
```

3.5 L2 (Euclidean) Normalisation

Each document vector is L2-normalised so that all vectors lie on the unit hypersphere. This means the cosine similarity between any two vectors is their dot product — a computationally simple operation:

```
norm = sqrt( sum( w(t,d)^2 for all t ) )  
v_hat(d) = v(d) / norm  
  
After normalisation: cosine(a, b) = dot(a, b) = sum( a_i × b_i )
```

3.6 Cosine Similarity

Cosine similarity measures the angle between two document vectors, not their magnitude. Two findings discussing the same risk area will use similar vocabulary, producing vectors pointing in the same direction (high cosine similarity), regardless of how long the texts are:

```
cosine(a, b) = sum( a_i × b_i )      (because vectors are already unit-normalised)  
  
Range: [-1, 1] → 1 = identical direction, 0 = orthogonal, -1 = opposite
```

3.7 Cosine Distance

For clustering (where we want distance, not similarity), the code uses:

```
cosine_distance(a, b) = 1 - cosine(a, b)
```

3.8 K-Means++ Initialisation

Standard K-Means suffers from poor random centroid seeds leading to bad local optima. K-Means++ selects initial centroids with probability proportional to the squared distance from already-chosen centroids:

```
P(x_i is chosen) ∝ min_c [ cosine_distance(x_i, c) ]  
  
This ensures seeds are spread across the feature space, dramatically  
improving convergence quality.
```

3.9 K-Means Inertia

After assignment, inertia measures total within-cluster variance — the sum of cosine distances from each point to its assigned centroid. The restart with minimum inertia is chosen:

```
inertia = sum_i [ 1 - cosine( x_i, centroid(label_i) ) ]
```

3.10 Bisecting K-Means

Rather than fitting k clusters simultaneously, Bisecting K-Means iteratively splits the worst cluster into two using 2-Means. This produces a hierarchical decomposition that is more stable and semantically coherent than flat K-Means, especially for unequally-sized clusters:

1. Start: all points in one cluster.
2. Select the largest cluster (with highest inertia as tiebreaker).
3. Run 2-Means on that cluster to split it into two sub-clusters.
4. Reject degenerate splits (one empty side).
5. Repeat until k clusters are formed.

3.11 Silhouette Score

Silhouette measures how well each point fits its assigned cluster vs. the nearest alternative cluster. It is used to evaluate candidate values of k :

```
For each point  $i$ :  
   $a(i)$  = mean intra-cluster distance      (cohesion – lower is better)  
   $b(i)$  = mean distance to nearest other cluster (separation – higher is better)  
   $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$   
  
Mean silhouette = average  $s(i)$  across all points  
Range:  $[-1, 1]$  → closer to 1 is better
```

3.12 Gap Statistic

The gap statistic compares the within-cluster dispersion of the real data against random (uniformly distributed) data. A high gap means the clustering captures genuine structure:

```
 $W(k)$           = within-cluster sum of pairwise cosine distances /  $(2 \times \text{cluster size})$   
 $W_{\text{ref}}(k)$    = same metric on  $n_{\text{refs}}$  random uniform reference datasets  
  
 $\text{Gap}(k) = E[\log W_{\text{ref}}(k)] - \log W(k)$   
  
Higher Gap → the real clustering is much tighter than random → good  $k$ 
```

3.13 Composite k Score

Both scores are normalised to $[0,1]$ and combined with a small compactness penalty to avoid over-fragmentation:

```
composite( $k$ ) =  $0.6 \times \text{norm\_silhouette}(k) + 0.4 \times \text{norm\_gap}(k) - 0.005 \times k$   
  
The optimal  $k = \text{argmax composite}(k)$ 
```


4. Step-by-Step Process Flow

Step	Action	Why / Key Decision
① Ingest	Finding records enter findings_db via API or seed data	Central mutable store; all ML runs against this list
② Weight Docs	Each finding's fields are repeated proportionally to FIELD_WEIGHTS	model_theme×8, description×8 dominate; remediation×1 barely influences
③ Tokenise	Lowercase → regex extract → remove STOP_WORDS	Eliminates grammatical noise; keeps domain vocabulary
④ Build IDF	Count df(t) across all documents	Identifies discriminative vs. generic terms corpus-wide
⑤ TF-IDF	Compute sublinear TF → BM25 normalise → multiply by IDF → L2 normalise	Each finding becomes a unit-length vector in vocabulary space
⑥ Auto-k	Run Silhouette + Gap across k range using Bisecting K-Means	Avoids hardcoding k; adapts to dataset size and theme diversity
⑦ Bisecting KM	Iteratively split largest cluster with 2-Means until k clusters formed	More stable than flat K-Means; respects natural theme boundaries
⑧ Sort	Clusters sorted by dominant model_theme against a canonical order	Stable, human-readable cluster IDs across re-runs
⑨ Annotate	Generate semantic_signals (top-10 tokens) and why_grouped prose	Gives reviewers a plain-English rationale for each cluster
⑩ Serve	Flask REST API exposes findings, clusters, search endpoints	Decouples ML logic from UI; enables future integrations
⑪ Search	Query vectorised alongside docs; cosine-ranked; best cluster found by overlap	Pure TF-IDF retrieval — no separate index needed

5. Detailed Line-by-Line Code Explanation

5.1 Imports and App Initialisation (Lines 1–7)

```
from flask import Flask, render_template, request, jsonify
import json, re, math
from collections import defaultdict

app = Flask(__name__, template_folder='.', static_folder='.', static_url_path='/')
```

flask imports	Flask = web framework; render_template = serve HTML; request = read incoming data; jsonify = convert dicts to JSON responses
json, re, math	json for potential serialisation; re for regex tokenisation; math for log, sqrt in TF-IDF formulas
defaultdict	A dict that auto-initialises missing keys to a default (e.g., int → 0). Used throughout for frequency counting.
app = Flask(...)	Creates the web app. template_folder='.': look for index.html in the same directory. static_url_path='/': serve static files (JS, CSS) from root URL.

5.2 In-Memory Data Stores (Lines 10–11)

```
findings_db = []
clusters_db = []
```

These two Python lists serve as the application's runtime database. findings_db holds raw finding records (dicts). clusters_db holds the output of the clustering algorithm. Both are global and mutated in-place. They are reset when the server restarts — there is no persistence to disk.

5.3 Stop Words (Lines 16–33)

```
STOP_WORDS = {'a', 'an', 'the', 'is', 'are', 'was', ...}
```

A hardcoded set of English function words and domain-generic verbs. The set is intentionally lean: words like 'model', 'risk', and 'finding' were deliberately excluded because they appear in every MRM document and would otherwise add noise without discrimination. Using a Python set gives O(1) membership tests.

5.4 Field Weights (Lines 42–48)

```
FIELD_WEIGHTS = {
    'model_theme': 8,
    'title': 5,
    'description': 8,
    'business_justification': 6,
    'suggested_remediation': 1,
}
```

These weights control how many times each field's text is repeated in the composite document (see `build_weighted_doc`). Higher repetition means those terms appear more often and earn higher TF values, making them more influential in the final TF-IDF vector. `model_theme` and `description` are co-equal dominant signals; `remediation` is nearly silenced because its action-language vocabulary is often generic across very different findings.

5.5 Theme Groups and `normalize_theme` (Lines 52–72)

```
THEME_GROUPS = {
    'modelling input data': 'data_quality',
    'model documentation': 'documentation',
    ...
}

def normalize_theme(theme):
    return THEME_GROUPS.get(theme.lower().strip(), theme.lower().strip())
```

MRM datasets often use slightly different spellings for the same theme (e.g., 'Modelling Input Data' vs 'Model Input Data'). `THEME_GROUPS` maps all variants to a single canonical key. `normalize_theme` applies this mapping at runtime. When `build_weighted_doc` repeats the `model_theme` 8 times, all variants resolve to the same canonical string, ensuring cosine similarity is not broken by spelling differences.

5.6 `tokenize()` — Text Normalisation (Lines 75–78)

```
def tokenize(text):
    text = text.lower()
    tokens = re.findall(r'\b[a-z][a-z\-\]{2,}\b', text)
    return [t for t in tokens if t not in STOP_WORDS]
```

<code>text.lower()</code>	Case-fold: 'Governance' and 'governance' become the same token
<code>re.findall(r'\b[a-z][a-z\-\]{2,}\b')</code>	Regex extracts alphabetic tokens of at least 3 characters (with optional hyphens). Discards numbers, punctuation, and single/double-character words.
<code>t not in STOP_WORDS</code>	Filters the token list against the stop-word set in $O(1)$ per token. Returns only semantically meaningful domain terms.

5.7 `build_weighted_doc()` — Field Repetition (Lines 81–94)

```
def build_weighted_doc(finding):
    parts = []
    for field, weight in FIELD_WEIGHTS.items():
        val = finding.get(field, '') or ''
        if field == 'model_theme':
            val = normalize_theme(val)
        parts.extend([val] * weight)
```

```
return ' '.join(parts)
```

This is the field-weighting mechanism. Rather than modifying the TF-IDF formula itself, the function exploits the TF calculation: repeating a field's text n times artificially increases the term counts for that field by factor n . For example, with `model_theme weight=8`, the theme string appears 8 times in the composite document — its tokens receive $8\times$ higher raw TF than they would from a single mention.

5.8 build_tfidf() — Vectorisation Engine (Lines 97–138)

```
def build_tfidf(docs, sublinear_tf=True, bm25_b=0.4):
    N = len(docs)
    tokenized = [tokenize(d) for d in docs]
    df = defaultdict(int)
    for toks in tokenized:
        for t in set(toks):      # set() counts each term once per doc
            df[t] += 1
    vocab = list(df.keys())
    word2idx = {w: i for i, w in enumerate(vocab)}
    V = len(vocab)
    avg_dl = sum(len(t) for t in tokenized) / max(N, 1)
```

First pass: builds the vocabulary. `tokenize()` is called on every composite document. Document frequency `df[t]` counts in how many documents each term appears (using `set()` to avoid double-counting within a single document). The vocabulary is indexed into `word2idx` for fast vector construction. `avg_dl` is the mean token count per document, needed for BM25 normalisation.

```
vectors = []
for toks in tokenized:
    dl = len(toks) or 1
    tf_raw = defaultdict(int)
    for t in toks:
        tf_raw[t] += 1
    vec = [0.0] * V
    for t, cnt in tf_raw.items():
        tf = (1 + math.log(cnt)) if sublinear_tf else (cnt / dl)
        tf_norm = tf / (1 - bm25_b + bm25_b * (dl / avg_dl))
        idf = math.log((N + 1) / (df[t] + 1)) + 1
        vec[word2idx[t]] = tf_norm * idf
    norm = math.sqrt(sum(x*x for x in vec)) or 1
    vectors.append([x / norm for x in vec])
```

Second pass: builds each document's TF-IDF vector. For every term in the document: (1) compute sublinear TF; (2) apply BM25 length penalty; (3) multiply by smoothed IDF. The resulting raw vector is then L2-normalised. The final vectors list contains one unit-length vector per document.

5.9 cosine() — Dot Product Similarity (Lines 141–142)

```
def cosine(a, b):  
    return sum(x * y for x, y in zip(a, b))
```

Because all vectors are L2-normalised (unit length), the cosine similarity of two vectors is simply their dot product. This is the most-called function in the codebase — it runs inside every K-Means iteration, silhouette calculation, and search query. It is intentionally kept as a pure Python one-liner for simplicity, accepting the performance trade-off vs. NumPy.

5.10 kmeans() — K-Means with Restarts (Lines 145–207)

```
def kmeans(vectors, k, max_iter=300, n_restarts=5, seed=42):  
    # k-means++ initialisation  
    centroids = [vectors[random.randint(0, n-1)][:]]  
    for _ in range(k-1):  
        dists = [min(1-cosine(v,c) for c in centroids) for v in vectors]  
        # sample next centroid with prob  $\propto$  distance  
        ...  
    # Assignment + centroid update loop  
    for _ in range(max_iter):  
        new_labels = [argmax cosine(v, c) for v in vectors]  
        if converged: break  
        recompute centroids; handle empty clusters  
    # Return best-inertia result across n_restarts
```

k-means++ init	Spreads initial centroids probabilistically. The first centroid is random; each subsequent centroid is sampled with probability proportional to its minimum cosine distance from existing centroids. Prevents degenerate clusterings.
Assignment step	Each vector is assigned to the centroid with maximum cosine similarity (minimum distance). This is the 'E' step of the EM interpretation.
Update step	Centroids are recomputed as the mean of their assigned vectors. If a cluster is empty, it is re-seeded with the point farthest from its current centroid ('stealing') to avoid wasted cluster capacity.
n_restarts=5	The entire K-Means loop (init + iteration) is run 5 times with different seeds. The run with minimum inertia is returned. Mitigates local minima sensitivity.
seed + restart*31	Deterministic seeds per restart: reproducible across server restarts while still exploring different initialisations.

5.11 bisecting_kmeans() — Hierarchical Splitting (Lines 210–270)

```
def bisecting_kmeans(vectors, k, seed=42):  
    clusters = [list(range(n))] # start: one cluster  
    while len(clusters) < k:  
        # Select cluster with largest size (inertia tiebreaker)  
        target_idx = max(..., key=lambda j: (len(clusters[j]), inertia(j)))  
        # Run 2-means on the selected sub-cluster  
        best_labels_2, _ = kmeans(sub_vecs, 2, ...)  
        # Split into group_a and group_b
```

```
# Reject degenerate splits
clusters.pop(target_idx)
clusters.append(group_a)
clusters.append(group_b)
```

Bisecting K-Means avoids the most common failure mode of flat K-Means: all centroids collapsing into the densest region of the feature space. By splitting one cluster at a time, each 2-Means sub-problem is small and tractable. Selecting the largest cluster (with inertia as a tiebreaker) prioritises the splits that will most reduce overall dispersion. Degenerate splits (one empty sub-cluster) are rejected and the cluster is left unsplit.

5.12 `_silhouette_score()` — Cluster Quality Metric (Lines 273–306)

```
def _silhouette_score(vectors, labels, k):
    for i, label in enumerate(labels):
        a = mean intra-cluster cosine distance for point i
        b = mean distance from i to nearest other cluster
        s(i) = (b - a) / max(a, b)
    return mean(s)
```

For each point, silhouette computes how much closer it is to its own cluster (a) versus the next-closest cluster (b). A score near 1 means the point is deep inside its cluster and far from others — ideal. Near 0 means the point is on a cluster boundary. Negative means it is closer to another cluster — a misassignment. The mean silhouette across all points is the evaluation metric for a given k.

5.13 `_gap_statistic()` — Random Baseline Comparison (Lines 309–352)

```
def _gap_statistic(vectors, labels, k, n_refs=5):
    W = within_cluster_sum_of_pairwise_distances(vectors, labels, k)
    for _ in range(n_refs):
        ref_vecs = random uniform unit vectors
        ref_labels, _ = kmeans(ref_vecs, k, ...)
        ref_logs.append(log(W_ref))
    return mean(ref_logs) - log(W)
```

The gap statistic answers: 'Is our clustering meaningfully better than random?' It clusters `n_refs` sets of random points with the same k and measures their within-cluster dispersion. If the real data's clustering is much tighter than the random baseline (high gap), k is a good choice. The reference vectors are uniform random points normalised to unit length (living on the same hypersphere as the real TF-IDF vectors) for a fair comparison.

5.14 `_auto_select_k()` — Optimal k Decision (Lines 355–432)

```
def _auto_select_k(vectors, n, has_themes, theme_count):
    # Determine k search range based on n
    if n <= 10: k_min, k_max = 2, n-1
```

```
elif n <= 50: k_max = min(12, n//2)
...
for k in range(k_min, k_max+1):
    sil_scores[k] = silhouette(bisecting_kmeans(vectors, k))
    gap_scores[k] = gap_statistic(vectors, labels, k)
# Normalise both to [0,1]
composite(k) = 0.6 * norm_sil + 0.4 * norm_gap - 0.005 * k
return argmax composite
```

The search range scales with dataset size to keep runtime manageable. If `model_theme` fields are present, `theme_count` informs the upper bound of the range — the algorithm expects at least as many clusters as distinct themes. Both silhouette and gap scores are min-max normalised to [0,1] so they are on the same scale before combining. The $0.005 \times k$ compactness penalty gives a mild tie-breaking preference for smaller `k` values, discouraging unnecessary fragmentation.

5.15 `run_clustering()` — Orchestration (Lines 488–543)

```
def run_clustering(k=None):
    global clusters_db
    has_themes = any(f.get('model_theme') for f in findings_db)
    docs = [build_weighted_doc(f) for f in findings_db]
    vectors, vocab, word2idx = build_tfidf(docs)
    if k is None:
        k = _estimate_k(vectors, n, has_themes)
    labels, _ = bisecting_kmeans(vectors, k)
    if not labels:
        labels, _ = kmeans(vectors, k) # fallback
    # sort clusters by dominant theme, annotate, store in clusters_db
```

`run_clustering` is the master orchestrator. It is called automatically every time `findings_db` changes (add, delete, bulk import). It rebuilds everything from scratch: weighted docs, TF-IDF vectors, `k` selection, clustering, and cluster metadata. The global `clusters_db` is replaced atomically. The fallback to standard K-Means exists for edge cases where bisecting K-Means returns no labels (e.g., degenerate input).

5.16 `search_query()` — Semantic Search (Lines 546–568)

```
def search_query(query):
    all_docs = docs + [query]
    vectors, ... = build_tfidf(all_docs)
    q_vec = vectors[-1]
    sims = sorted([(cosine(q_vec, dv), i) for i, dv in enumerate(doc_vecs)],
reverse=True)
    top_findings = [findings_db[i] for _, i in sims[:5]]
    # find cluster with most top-5 findings
    best_cluster = max(clusters, key=overlap_count)
```

Search appends the query to the document corpus, rebuilds the entire TF-IDF matrix (so the query is vectorised in the same vocabulary space as the documents), and computes cosine similarity

between the query vector and every document vector. The top-5 findings are returned. The best cluster is then identified as the one with the highest overlap against those top-5 findings — a cluster-level relevance signal without needing a separate cluster search index.

5.17 Flask REST API Routes (Lines 572–751)

Route	Method	Purpose
GET /	GET	Serves index.html (the SPA frontend)
GET /api/findings	GET	Returns all findings as JSON array
POST /api/findings	POST	Adds a single finding; re-runs clustering
POST /api/findings/bulk	POST	Adds many findings from a JSON array
POST /api/findings/upload	POST	Parses an Excel/CSV file and bulk imports
DELETE /api/findings/<fid>	DELETE	Removes a finding by ID; re-runs clustering
GET /api/clusters	GET	Returns all clusters with metadata
POST /api/clusters/rerun	POST	Manually triggers clustering (optional k)
POST /api/search	POST	Runs semantic search; returns top findings + cluster
GET /api/stats	GET	Returns total count of findings and clusters

5.18 upload_findings() — File Import (Lines 626–714)

```
if filename.endswith('.xlsx'):
    df = pd.read_excel(io.BytesIO(file.read()))
elif filename.endswith('.csv'):
    df = pd.read_csv(io.BytesIO(file.read()))

# Flexible column resolution
col_map = { col.lower().replace(' ','_'): col for col in df.columns }
finding_id_col = resolve(['finding_id', 'findingid', 'finding_no', 'id'])

# Skip missing required fields and duplicate IDs
for _, row in df.iterrows():
    if not fid or fid.lower() == 'nan': skipped += 1; continue
    if any(f['finding_id'] == fid for f in findings_db): skipped += 1; continue
    findings_db.append({...})
```

File bytes are read into an in-memory buffer (`io.BytesIO`) — no temp file is written to disk. Column names are normalised to lowercase_with_underscores and matched against a list of accepted variants for each required field. This makes the import robust to minor header variations in user-uploaded spreadsheets. Rows with missing required fields or duplicate IDs are silently skipped and counted.

5.19 Seed Data and Application Startup (Lines 754–824)

```
SAMPLE_FINDINGS = [ { 'finding_id': 'F001', ... }, ... ] # 12 pre-built findings

for f in SAMPLE_FINDINGS:
    findings_db.append(f)
run_clustering()

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

At module load time (whether run directly or via gunicorn), the 12 sample findings covering all 6 MRM themes are pre-loaded into `findings_db` and clustering is immediately run. This ensures the application is always functional on first load without requiring any user action. The `if __name__ == '__main__':` guard means the development server (`debug=True`) only starts when the script is run directly; when deployed via gunicorn, Flask is loaded as a WSGI app without launching the dev server.

6. Deployment Architecture

Component	Role	Configuration
Flask	WSGI application framework; handles HTTP routing, request parsing, JSON responses	<code>app = Flask(__name__, ...)</code>
Gunicorn	Production-grade WSGI server; multi-worker process manager for concurrency	Listed in requirements.txt; invoked via <code>gunicorn index:app</code>
Pandas + OpenPyXL	Excel/CSV parsing for file uploads	Imported lazily inside the upload route to avoid startup overhead
index.html	Single-page frontend; served as a static template from the same directory	<code>template_folder='.'</code> in app init
In-memory state	<code>findings_db</code> and <code>clusters_db</code> live in Python process memory	Resets on restart; suitable for single-process deployments

7. Summary

This system implements a self-contained, NumPy-free NLP clustering pipeline specifically tuned for Model Risk Management audit findings. Its key design decisions are:

- Field weighting via text repetition: avoids modifying the TF-IDF formula while achieving domain-specific emphasis on `model_theme` and description.
- Sublinear TF + BM25 normalisation: prevents high-frequency domain jargon from dominating; equalises influence of short and long fields.
- Bisecting K-Means: produces more stable, semantically coherent clusters than flat K-Means on a dataset with known thematic structure.
- Combined Silhouette + Gap statistic: automatically determines `k` without requiring user input, adapting to corpus size and theme diversity.
- TF-IDF search at query time: appends the query to the corpus and rebuilds vectors, ensuring the query is compared in the exact same vocabulary space as documents.
- REST API + SPA frontend: clean separation of ML logic from presentation; the frontend is a fully independent HTML/JS client consuming the JSON API.

The result is an application that loads pre-seeded findings, clusters them automatically, and exposes a full API for real-time finding management, semantic search, and cluster exploration — all in approximately 820 lines of pure Python and standard library code.